



University of Bahrain
College of Information Technology
Department of Information Systems
B.Sc. In Cybersecurity

Thaghrah: Practical Cybersecurity Learning Platform Through Packet Analysis

Prepared by

Manar Hussain Abdulrahim 202201554

Noora Abdulrahman Bumjaid 202203245

For

ITCY 499

Senior Project

Academic Year 2025-2026 Semester 2

Project Supervisor:

Dr. Abdulla Khalifa Aldoseri

12 May 2026

ABSTRACT

The transition from theoretical networking concepts to real-world packet analysis using packet analyzer tools such as Wireshark is challenging for many students starting out in cybersecurity, and this is because of the lack of practical experience with realistic traffic investigations. To solve this, a web-based learning platform named Thaghras has been built by implementing packet analysis and network traffic investigation with Flask, SQLite and Scapy. There are downloadable PCAP challenges on the platform, with several features aimed at gamification, and challenges from various protocol categories. An integrated AI Lab enables users to create full custom scenarios from user prompt via workflows written in Scapy. The results demonstrated that the practical activities enhanced the engagement and understanding of the learners, as opposed to the theory-based approach, while the scenarios created by AI lessened repetition and provided diverse investigations in a controlled educational setting.

ACKNOWLEDGMENTS

We would like to first praise Allah for the strength and patience he blessed us with in the completion of this work. There were difficult stages during development, but with His guidance we were able to continue and finish the work successfully.

Also, our gratefulness to our supervisor Dr. Abdulla Khalifa Aldoseri for his guidance, support, and patience throughout the project. His feedback helped us improve many parts of the system, especially during development and testing. We also benefited from his advice during the technical and academic stages of the project.

We would also like to extend our gratitude to our families and friends for their support and understanding throughout the process. Their support helped us stay motivated during stressful periods and long working hours.

Finally, working on Thaghrah gave us valuable practical experience in cybersecurity, web development, and packet analysis. The planning, implementing, testing and refining phases of the project required a lot of time and effort, and we felt that this reinforced our technical skills and teamwork.

Table of Contents

ABSTRACT	i
ACKNOWLEDGMENTS	ii
LIST OF TABLES	vi
LIST OF FIGURES	vii
Chapter 1 Introduction.....	1
1.1 Problem Statement	1
1.2 Project Objectives.....	2
1.2.1 Scope.....	2
1.2.2 Target Audience	3
1.2.3 Impact.....	3
1.2.4 Limitations	3
1.2.5 Assumptions	3
1.3 Contribution	4
1.4 Report Outline	4
Chapter 2 Literature Review	5
2.1 Background.....	5
2.2 Related Work	5
2.3 Summary	6
2.4 Relevance of Thaghras	7
Chapter 3 Project Management.....	8
3.1 Process Model	8
3.2 Risk Management.....	9
3.3 Project Activities Plan.....	10
3.4 Challenges Faced During Development	11
Chapter 4 Required Elicitation and Analysis.....	13
4.1 Requirement Elicitation	13
4.2 System Requirements.....	14
4.2.1 Functional Requirements	14
4.2.2 Non-Functional Requirements	15
4.3 System Models.....	16

4.3.1 System Architecture	16
4.3.2 Entity Relationship Diagram	17
Chapter 5 Required Elicitation and Analysis.....	18
5.1 Introduction	18
5.2 Database Schema Design.....	18
5.2.1 Purpose of the Schema in the System	18
5.2.2 Data Dictionary	18
5.3 User Interface Design.....	23
5.3.1 Login and Registration Interface	23
5.3.2 Home Dashboard Interface.....	25
5.3.3 Challenge Interface	26
5.3.4 AI Lab Interface	28
5.3.5 Protocol Guide Interface	30
5.3.6 Surprise Interface	31
5.4 Navigation and User Flow	31
5.5.1 AI PCAP Packet Construction	32
5.5.2 AI Challenge Generation	33
5.5.3 Calling LLM and Validating Challenge Payload.....	34
5.5.4 Flag Validation	35
5.5.5 Flag Normalization	36
5.5.5 Challenge Unlock Progression	36
5.5.6 Flag Submission	37
5.5.7 Challenge Completion	38
5.6 Summary	39
Chapter 6 System Implementation and Testing.....	40
6.1 System Implementation.....	40
6.1.1 Development Environment and Hardware Specifications	40
6.1.2 Technology Stack Selection and Justification	41
6.1.3 Project Structure and Module Organization.....	42
6.2 Evaluation and Testing	43
6.2.1 Testing Methodology	43
6.2.2 Functional Testing	44

6.2.3 Security Testing	48
6.2.4 Performance Testing	53
6.2.5 AI Accuracy Testing.....	54
6.2.6 Discussion, Comparison, Strengths, and Weaknesses	55
6.3 Summary	57
Chapter 7 Conclusion and Future Work	58
References	59

LIST OF TABLES

Table 1: Identified Risks and Mitigation Strategies for Thaghrah	9
Table 2: Project Activities Plan for Thaghrah	10
Table 3: Requirement Elicitation Responses	13
Table 4: Users Table Metadata	18
Table 5: Challenge Categories Table Metadata	19
Table 6: Challenges Table Metadata	19
Table 7: User Progress Table Metadata	20
Table 8: Badges Table Metadata	21
Table 9: User Badges Table Metadata	21
Table 10: AI Challenges Table Metadata	22
Table 11: Hardware Configuration (Development and Testing Workstation)	40
Table 12: Technology Stack Overview	41
Table 13: Functional Testing Results	44
Table 14: Security Testing Results	48
Table 15: AI Accuracy Testing Results	54
Table 16: Strengths of Thaghrah	56
Table 17: Weaknesses of Thaghrah	56

LIST OF FIGURES

Figure 1: Waterfall Process Model Diagram	8
Figure 2: Project Gantt Chart	11
Figure 3: System Architecture Diagram	16
Figure 4: ERD Diagram	17
Figure 5: Welcome Page	23
Figure 6: Registration Page	24
Figure 7: Login Page	24
Figure 8: Home Dashboard Page	25
Figure 9: Challenge Navigation Page	26
Figure 10: Protocol Challenges Page	27
Figure 11: Challenge Page	27
Figure 12: AI Challenge Page	28
Figure 13: AI Challenge Automatic Prompt Page.....	28
Figure 14: AI Challenge Generated Challenge Page	29
Figure 15: Guide Page.....	30
Figure 16: Cheat Sheet Page	30
Figure 17: Surprise Page	31
Figure 18: User Flow Diagram.....	31
Figure 19: Project Structure	43
Figure 20: PCAP Download Validation.....	45
Figure 21: Flag Validation Testing.....	45
Figure 22: Hint Penalty Validation	46
Figure 23: Badge Awarding Validation	47
Figure 24: AI Challenge Generation Validation	47
Figure 25: Password Hashing in Database	49
Figure 26: Password Hashing in Logs	49
Figure 27: XSS Script Injection	49
Figure 28: XSS Input Handling Validation	50
Figure 29: URL Access Attempt	50
Figure 30: Locked Challenge Access Protection	51
Figure 31: Weak Credentials Validation	51
Figure 32: Weak Credential Validation	52
Figure 33: Weak Credential Validation	52
Figure 34: Browser Time	53

Figure 35: Log Time.....	53
Figure 36: AI Accuracy Testing Results	55

Chapter 1

Introduction

This chapter introduces the foundation of the project by presenting the background and motivation behind the development of Thaghrah. It introduces the problem that the proposed system aims to solve and why an interactive system which enables packet-analysis learning in cybersecurity education is needed. The chapter also defines the project objectives, scope, and significance, outlines the proposed solution at a high level, and concludes with the overall structure of the report to provide an overview of the chapters that follow.

1.1 Problem Statement

Packet analysis and traffic investigation are difficult areas for many beginner cybersecurity students. A learner may understand protocols such as TCP, DNS, or ICMP during lectures and still struggle when trying to analyze those protocols inside a real packet capture. The theory may seem clear in class, but applying it in Wireshark is different, and this is usually where many beginners start facing problems (Orebaugh, et al., 2006).

A single PCAP file can contain many things happening at once. Learners may need to inspect packet structures, understand protocol behavior, follow communication streams, apply filters, and notice suspicious traffic patterns at the same time. For someone with limited practical experience, the amount of information can become overwhelming very quickly. Another issue appears when learners move from theory into practical investigation, as some students are unfamiliar with filtering packets, inspecting protocol fields, or recognizing abnormal traffic inside PCAP files. Because of this, learners may memorize networking concepts for exams without developing the practical investigation skills expected in cybersecurity environments.

Many educational environments also provide limited hands-on packet-analysis practice. Some courses spend more time explaining concepts than allowing learners to investigate traffic themselves. Students may also move to more advanced topics before becoming comfortable with earlier analysis steps, making later investigations harder than they should be.

There is also a replayability problem in many packet-analysis exercises. Once learners solve a fixed challenge, the investigation process and expected answer become familiar, making future attempts

predictable and reducing learning value. For these reasons, there is a need for a beginner-friendly learning environment that provides guided practice with realistic PCAP files, protocol-focused exercises with gradually increasing difficulty, and diverse scenarios to build confidence and practical traffic-analysis skills.

1.2 Project Objectives

Thaghray's primary goal is to teach beginner cybersecurity students how to enhance their packet analysis and network traffic investigation skills by using hands-on practice, instead of just theory. Many of the beginners have difficulties moving from concepts to real packet analysis in Wireshark, so the platform is built around the practical investigation process using downloadable PCAP challenges. These challenges are presented in a progressive sequence, so that learners can begin with less complex challenges and work up to more complex protocol-analysis challenges. Thaghray also features an AI Lab, which will create packet investigation scenarios based on user prompts to increase the variety of the training, without changing the solve-and-submit workflow. There are also progress tracking components and unlocking progression to make sure that there is ongoing engagement.

1.2.1 Scope

This project is targeted at beginner and intermediate level cybersecurity learning, in which users are provided downloadable PCAPs that are analyzed locally using Wireshark. There are primarily 8 protocol categories, which are HTTP, TCP, DNS, FTP, ICMP, SMTP, TLS, and network forensics. The platform includes core user features such as authentication and progress persistence using SQLite. It contains 40 pre-defined investigation challenges, with 5 challenges assigned to each protocol. They are arranged in increasing order of difficulty and unlocked based on completion, allowing learners to progress gradually rather than being given access to all challenges at once. This is a structure put together to help learners become more comfortable at using Wireshark and traffic analysis.

There are also a variety of additional features such as flag submission, score tracking, badges, challenge completion progress tracking, hints that deduct points, and protocol-reference guides. An AI Lab is also included that automatically creates one-time custom investigation scenarios using the Grok API with the ability to generate downloadable PCAP files using the included Scapy traffic generation for analysis in the same learning environment.

1.2.2 Target Audience

Thaghras is focused on cybersecurity students and beginners wanting to gain hands-on experience in the fields of packet analysis and network investigation. It is designed for university students in cybersecurity, networking, digital forensics, and information security, including anyone who would like to gain hands-on experience with Wireshark. The platform could also enable instructors and educational institutions to teach with the platform by having an organized learning space with challenges to download, and a linear progression for practical cybersecurity training.

1.2.3 Impact

The platform provides an opportunity to practice analytical thinking and protocol comprehension with real-life scenarios, enhancing the learning experience. The guided challenge structure facilitates effective learning by giving clear instructions and hints to help learners overcome challenges without confusion. The use of AI-generated challenge creation also highlights the potential of AI in education to enhance diversity and engagement. Moreover, the platform serves as a basis for future educational systems, which will integrate intelligent content generation and practical security training (Kasneci, et al., 2023).

1.2.4 Limitations

Thaghras is limited to offline use through downloadable PCAP files and does not support live packet interception or real-time network monitoring. It is designed for educational and local deployment purposes using SQLite rather than large-scale enterprise use. There are also limitations in the AI Lab area, as generated scenarios are based on the Grok API and Scapy-based workflows, which can be impacted by its availability, inconsistent responses, or malformed outputs. Moreover, challenges created through AI are not permanently saved and are only active while the server is running.

1.2.5 Assumptions

It is assumed that generated PCAP files are only utilized in a controlled learning environment, for authorized cybersecurity learning activities. It is also dependent on external tools such as Wireshark, which are required in order to properly analyze and interpret packet captures, as traffic is inspected away from the web application. The AI challenge-generation workflow relies on the assumption that the response received is structured, compatible, and can be processed by the

platform's validation logic. Additionally, successful deployment assumes that required API keys, dependencies, environment variables, and storage configurations are properly set up.

1.3 Contribution

1. Thaghras offers a dedicated educational environment through guided challenges and focuses particularly on traffic analysis learning.
2. The platform supports AI-generated challenges that enable learners to develop scenarios using natural-language prompts, rather than just preset challenges.
3. Interactive learning elements like score tracking, badges, milestone rewards, and hint penalties were added to engage and motivate learners.
4. Generated scenarios introduce a variety of tasks such as encoded payloads, fragmented traffic, encryption -based investigations, and brute-force simulations.
5. Challenges are presented in a progressive sequence so that users are able to develop investigation skills before moving on to more difficult challenges.
6. Integrating fixed challenges with dynamic AI challenges provides a self-contained learning environment for practice, testing and demonstration.

1.4 Report Outline

This report is organized as follows. The literature review is presented in Chapter 2, where concepts, existing systems and related technologies are discussed. Chapter 3 details the project methodology and development approach, and Chapter 4 details the system functional and non-functional requirements. Chapter 5 covers the system design, architecture, workflows and interfaces. The implementation and evaluation of the system are given in Chapter 6, which also includes the testing method and analysis of the results. Finally, a summary of the report and recommendations for future work are given in Chapter 7.

Chapter 2

Literature Review

This chapter focuses on background concepts associated with practical packet analysis and protocol-based training environments. It also looks at similar platforms such as OverTheWire, TryHackMe, and Burp Suite. The chapter also analyzes the increasing importance of hands-on learning and its shortcomings in current learning environments and how these observations shaped the development of Thaghrh.

2.1 Background

Cybersecurity education has evolved from purely theoretical teaching to more hands-on environments, especially in network security, where students are expected to analyze packet captures, rebuild traffic streams, and find evidence. This led to the widespread use of tools like Wireshark in education as well as industry, however, quality in learning is not just about the tool but the exercise quality, challenge design, and repeated practice (Orebaugh, et al., 2006).

Previous training for packet analysis used lab exercises that were static in nature and manually created PCAP files. These were useful for introducing networking concepts but tended to become predictable when used repeatedly as some learners memorized investigation patterns rather than build analytical skills. Manual creation was also time-consuming, especially with multiple protocol categories or forensic scenarios. In addition, structured investigation workflows and guided exercises began to appear in cybersecurity learning platforms, as did easier tools for creating automated packets like Scapy.

More recently, AI added a new dimension by allowing the creation of investigation scenarios based on natural language prompts to allow for more variation in training as well as creating challenges with validation and traffic quality (Kasneci, et al., 2023).

2.2 Related Work

OverTheWire is a free cybersecurity learning platform offering progressive wargames that use SSH covering Linux usage, shell scripts, privilege escalation, and web security concepts (Chapman, et al., n.d.). The platform facilitates progressive challenges in guided technical learning and command (in, et al., 2018)-line problem solving. However, it is not about traffic

investigation, (Chapman, et al., n.d.) and its support for protocol analysis workflows is limited compared to an environment that is dedicated to packet analysis training.

TryHackMe offers web-based forensics, defensive security, penetration testing, and networking training rooms. It contains beginner-friendly guided learning paths and exercises using Wireshark. There are, however, still investigation activities that remain static and challenges can become predictable after repeated use (in, et al., 2018) (PortSwigger, 2024).

Burp Suite is mainly designed for web-security testing and HTTP/HTTPS traffic inspection. It enables you to monitor web vulnerabilities, analyze sessions, modify traffic and intercept requests via proxy monitoring (PortSwigger, 2024). It features powerful tools for analyzing web traffic, but it is focused on web application security rather than a wider protocol-based investigation.

While these platforms offer great practical training in cybersecurity, they vary in their support for protocol-diverse traffic analysis, dynamic challenge generation and guided workflows.

2.3 Summary

As a result of concepts and platforms reviewed, development of the project had several important observations:

- Static exercises can be helpful in introducing packet analysis concepts, but repeated structures could decrease training variety over time and become predictable.
- Sequential progression and guided learning paths enable learners to acquire skills in a gradual manner and not be introduced to higher level tasks at an early stage.
- Manual generation of realistic traffic requires a lot of development effort and maintenance complexity, particularly if several protocol types and forensic scenarios are encountered.
- Scapy provides simple tools to generate protocol-specific traffic and other simulated scenarios.
- Many current cybersecurity learning platforms are more geared towards general security training, web exploitation, or virtual-machine environments, and not towards dedicated network-traffic investigation workflows.
- AI generated challenges add dynamic scenarios, but the output needs to be validated to ensure traffic quality and protocol consistency.

2.4 Relevance of Thaghr

Thaghr's implementation decisions were affected by the reviewed platforms and background concepts. The platforms of OverTheWire and TryHackMe emphasized the importance of gradual learning and progressive unlocking, which resulted in the development of a fixed curriculum with 40 investigation challenges divided into 8 categories of protocols, where each one is unlocked in a sequential manner to support progressive learning.

The drawbacks of manual environment creation also played a part in designing the system, as the ability to use Scapy-based packet-generation workflows was incorporated to create scenarios like redirected TCP streams, DNS and ICMP exfiltration traffic, and even encoded payloads were all achievable via reusable scripts. The AI-generation pipeline was developed with the help of AI-assisted concepts. The platform handles natural language prompts creating scenarios using the Grok API, which then validates, protocol-aware parses, repairs, generates responses at run time, and cleans up for downloadable challenges. Some of the broad prompts produced partial or imperfect outputs and so to enable more consistent generation and retain the same solve-and-submit workflow as in fixed prompts, buttons were added to the prompts.

Measurable, structured training workflows, as highlighted in the reviewed systems, also helped shape aspects of implementation, including SQLite-based challenge synchronization, hint-based reduction of points, validation of answers in a normalized way, a lifecycle management of the AI-PCAPs, and request logging in the Flask-based system.

Chapter 3

Project Management

This chapter describes the project management tasks that were undertaken in the building of Thaghrah. It covers the process model used during implementation, the various risks identified during development and how to mitigate them. The chapter also includes the project activity plan and some of the technical issues encountered during platform construction.

3.1 Process Model

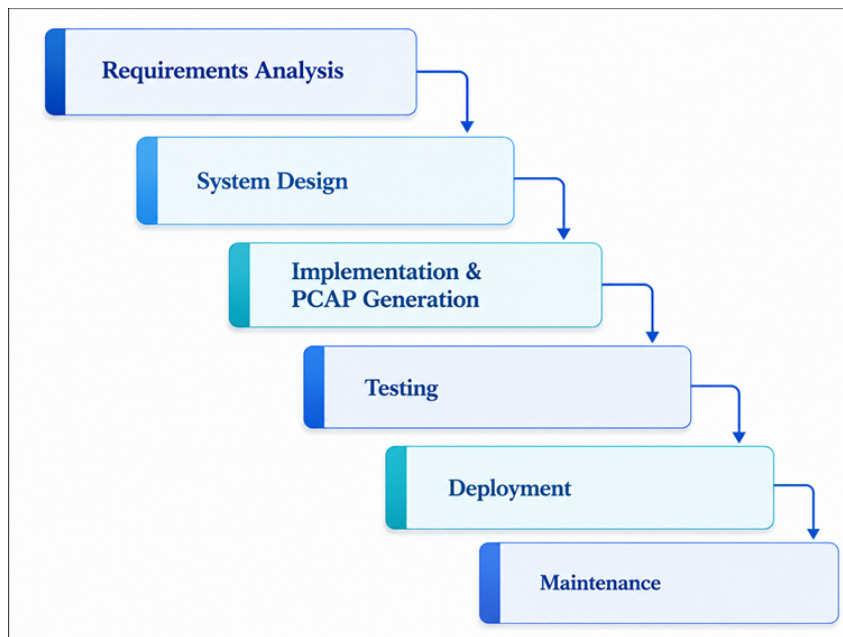


Figure 1: Waterfall Process Model Diagram

The Waterfall process model was used in the development of Thaghrah, as shown in Figure 1. It consisted of a sequence of steps, where each step was completed before proceeding to the next, since the main system requirements, challenge structure and the platform components were established early in the development.

The process began with requirements analysis, where educational objectives, and core system functions were identified. Next was the design stage, where the platform architecture, database structure, challenge organization and user interface were designed.

The implementation and PCAP-generation phase took the longest, involving the creation of the Flask application, SQLite database, the frontend elements, authentication system, and PCAPs for challenges. Several changes had to be made multiple times, especially regarding traffic behavior and functionality of AI elements.

Testing was then done on authentication, challenge unlock, validation of the flag, PCAPs, generated scenarios by the AI, and scoring. To achieve consistent behavior, multiple testing was required to solve problems. Following the test, the platform was made ready for deployment and further refined, as well as fixed bugs and maintained the overall system.

3.2 Risk Management

Thaghras presented a number of technical and operational challenges because of the combination of AI, with automated packet generation and challenge workflows controlled by the backend in the same environment. One of the most important considerations was the consistency in traffic generated as well as the challenge setup, since a slight mistake in the dynamic captures could disrupt the scenario. Some of the risks identified were malformed AI responses, invalid packet structures, API availability issues, and runtime cleanup failures.

To overcome these problems, gradual validation and monitoring was introduced, such as logging at runtime, temporary file management isolation, and challenge verification procedures. Stable platform behavior had to be attained through several refinements of some solutions before testing. Table 1 lists the identified risks, their severity and their mitigation measures.

Table 1: Identified Risks and Mitigation Strategies for Thaghras

Risk Description	Level	Mitigation Strategy
Malformed AI-generated challenge responses	High	Apply structured JSON parsing, response validation, and controlled repair workflows before challenge processing.
Invalid or corrupted generated PCAP files	High	Validate generated packet structures using Scapy-based protocol checks and controlled packet-generation scripts.
Incorrect protocol behavior during traffic generation	Medium	Use protocol-aware traffic-construction workflows and repeated testing for generated forensic scenarios.
Accumulation of temporary AI-generated PCAP files during runtime	Medium	Use isolated runtime storage and automatic cleanup mechanisms to remove generated files after server shutdown.

API failure or unavailable responses from the Grok API	Medium	Validate API responses before processing and prevent incomplete challenge generations during failures.
Incorrect challenge-answer validation	Medium	Apply normalized answer comparison and backend-controlled validation workflows.
Runtime logging and request-tracking failures	Low	Implement centralized logging mechanisms and request-timing monitoring during system execution.

3.3 Project Activities Plan

The project activities plan details the key activities that will be completed during the project's development cycle to make sure the project is developed in a structured manner.

Table 2: Project Activities Plan for Thaghrah

Phase	Main Activities	Start	End	Duration
Requirements Analysis	Identifying educational objectives, challenge requirements, packet-analysis workflows, AI Lab requirements, and non-functional requirements through research and requirement elicitation	15 Sep 2025	5 Oct 2025	~3 weeks
System Design	Designing the Flask architecture, database structure, ERD, user workflows, challenge organization, and system-interaction models	6 Oct 2025	9 Nov 2025	~5 weeks
Implementation & PCAP Generation	Developing the Flask application, frontend components, authentication system, scoring workflows, AI integration, and generating fixed and AI-based PCAP challenges using scripted workflows	10 Nov 2025	15 Mar 2026	~18 weeks
Testing	Testing authentication, challenge unlocking, flag validation, PCAP downloads, AI-generated scenarios, scoring mechanisms, and overall system functionality	16 Mar 2026	19 Apr 2026	~5 weeks
Deployment & Documentation	Preparing the system for demonstration and usage through environment configuration, static-file preparation, and project documentation	20 Apr 2026	3 May 2026	~2 weeks
Maintenance & Handover	Correcting remaining issues, synchronizing challenge data and scripts, refining content, and preparing the final project handover	4 May 2026	11 May 2026	~1 week

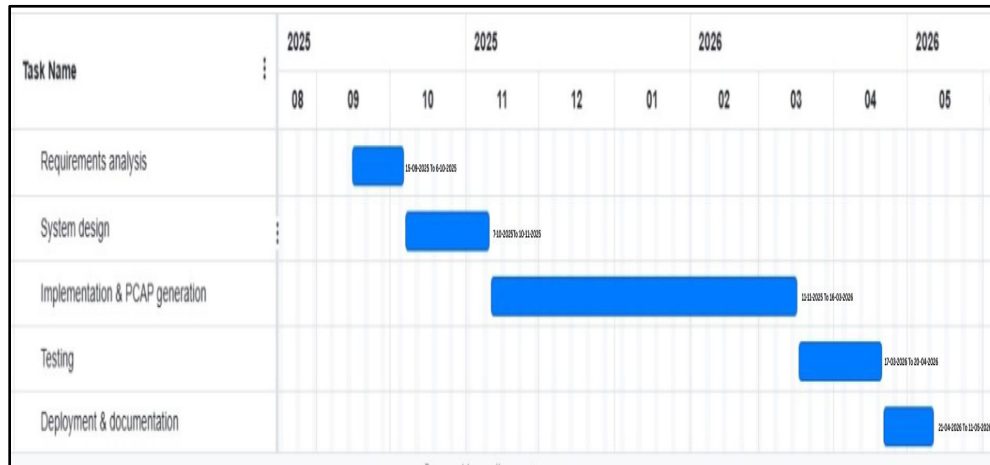


Figure 2: Project Gantt Chart

The project timeline in Figure 2 shows the visual sequence of the project development activities, following the Waterfall model in linear progression. The duration is between September 2025 and May 2026, and reflects the time allocated to various key project phases. The implementation and PCAP-generation phase took the longest time as it involved challenge development and integration of AI workflows. The latter phases involved system testing, deployment readiness, and documentation to guarantee project success.

3.4 Challenges Faced During Development

While developing the application Thaghrhah, a number of technical and implementation issues emerged. A significant challenge was balancing the difficulty level of the challenges. The platform was designed to begin with basic, simple tasks and gradually add complexity based on the various categories. Some challenges were too simple and could only be solved without any meaningful investigation, others were too difficult for beginners and involved more protocol knowledge than was anticipated. Because of this, many challenges went through repeated adjustments during development. Locations of the flags, traffic patterns and the level of difficulty were adjusted multiple times to ensure a seamless flow through the platform.

The AI-generated challenge workflow added a set of new issues. In early testing, the responses sometimes included incomplete or inconsistent challenge information. One problem was ensuring that the PCAP files contained the packets and traffic behavior specified in the prompt. In some

fragmentation scenarios, for instance, the whole flag was put into a single packet rather than distributed properly over fragmented traffic.

Based on this, extra validation logic and controlled parsing flows were added prior to generating output that would be converted into a PCAP file. One difficulty was hint generation, where some hints did not match the scenario, and others gave hints that were not clear enough, which meant that additional changes would have to be made to keep them in sync.

Chapter 4

Required Elicitation and Analysis

The requirement elicitation and analysis process to Thaghras is presented in this chapter in which educational and technical requirements were identified and converted into system functionalities and design models to be implemented in the platform. It also describes the functional requirements, non-functional requirements and the primary system models that are created for the project.

4.1 Requirement Elicitation

Based on the literature review, the requirement identification of Thaghras was done by observing the learning challenges of beginners and informal interviews with cybersecurity experienced individuals. The elicitation process centered on how understanding hands-on learning, student engagement, and exposure to tools affect student confidence and understanding compared to theory-only learning. Results indicated that many beginners have knowledge of cybersecurity concepts in theory but lack practical experience in real packet analysis workflows and investigation tasks.

The process also revealed the importance of guided exercises and gamification to keep learners motivated and to facilitate the gradual acquisition of skills (Hamari, et al., 2014). From these results, a series of features were designed into Thaghras, such as sequential progression, reference guides, cheat sheets, badges, and an AI Lab.

Table 3: Requirement Elicitation Responses

Question	Participant 1 Answer (Wants to remain anonymous)	Participant 2 Answer (Rashid Alnoaimi - Cybersecurity Specialist)
How well do beginner students understand cybersecurity concepts through theory alone?	Students understand the foundations with little or no practical experience.	Theoretical networking knowledge is important, but practical understanding is necessary to fully understand network security and infrastructure behavior.
Do first-year students receive enough exposure to real cybersecurity tools?	Students may learn about tools without getting enough hands-on usage opportunities.	Universities often focus more on theory before introducing practical applications and real-world environments.

How important is hands-on experimentation in cybersecurity education?	Hands-on practice combined with theory helps students apply concepts with understanding.	Practical experimentation is extremely important for understanding packet analysis, attacks, and network behavior.
What impact does limited practical experience have on students?	Without continuous practice, students may struggle to think critically and solve problems.	Students may feel uncertain when interacting with real systems and unfamiliar scenarios despite understanding concepts academically.
How useful are guided exercises for beginners?	Guided exercises are useful to some extent.	Step-by-step guidance helps learners understand complex tasks gradually and improves practical understanding.
What is your opinion on gamified cybersecurity learning?	Gamification can be useful occasionally.	Gamified environments improve engagement, motivation, participation, and learner interest in cybersecurity training.

4.2 System Requirements

This section outlines the functional and non-functional requirements of Thaghrah, outlining the core behavior, system functions and operational characteristics of the platform that are necessary to support challenge management and learner interaction.

4.2.1 Functional Requirements

- **FR-1 AI Challenge Generation**

The AI Lab produces challenges based on the user's input. They consist of a title, challenge description, downloadable PCAP, hint, and flag.

- **FR-2 Interactive PCAP Generation**

The platform has downloadable PCAPs for both fixed and AI challenges. Fixed challenges use a pre-generated set of packets generated using scripts. AI challenges dynamically create traffic according to the scenario and the prompt entered. Some generated traffic patterns also change between scenarios, which makes the investigation process less repetitive.

- **FR-3 Sequential Challenge Unlocking**

The challenges are presented in a sequence structure. Learners are not allowed to access a challenge until the previous has been successfully completed. This is to enable gradual skill development and to prevent immediate access to more challenging tasks.

- **FR-4 Flag Generation and Validation**

Packet captures have flags embedded within them in various ways which include fragmented traffic, redirection payloads, and image-based flags. The validation process is automatic and checks if the flag entered is accurate.

- **FR-5 Protocol Guides and Cheat Sheets**

Integrated protocol guides and cheat sheets are provided to help learners during investigations. The guides provide overview information about each protocol, and the cheat sheet provides a list of commands that could be used during the challenges.

- **FR-6 Scoring and Badge Awarding**

Points and badges are given based on challenges completed and progress. These have been included to promote learning engagement and participation in the platform.

- **FR-7 Progress Tracking**

Learner progress, completed challenges, earned points, and awarded badges are kept within the database system. This allows tracking of their progress

4.2.2 Non-Functional Requirements

- **NFR-1 Security**

The platform validates challenge access to prevent users from bypassing sequential progression through direct URL traversal, where it was tested to make sure they could not be accessed before completing the required progression order. User passwords are securely stored as Werkzeug password hashes and checked at login time with a secure hash comparison (Foundation, 2023). The flags submitted are also stored in the logs as fingerprints or lightweight metadata. This helps to minimize the risk of challenge flags being made public via runtime logs or debugging processes.

- **NFR-2 Portability**

The interface of the platform is responsive and adjusts to screen sizes. It can be accessed on the web browser and mobile device, without the need for a mobile app.

- **NFR-3 Usability**

Custom CSS and responsive layout patterns were used to design the interface, rather than third-party CSS framework. The platform's styling, navigation and interaction on the dashboard are consistent and structured, enabling investigations to be carried out in a usable way.

- **NFR-4 Maintainability**

The platform adopts a Flask architecture, consisting of separate modules for routings, content, AI services, and domain logic (Grinberg, 2018). Challenge definitions are stored as Python modules that are synced from the database via a separate synchronization script. The centralized schema structure also facilitates easier updates between components.

4.3 System Models

This section introduces the primary models of the system used to represent the structure and organization of Thaghras. It comprises of System Architecture Diagram and Entity Relationship Diagram (ERD).

4.3.1 System Architecture

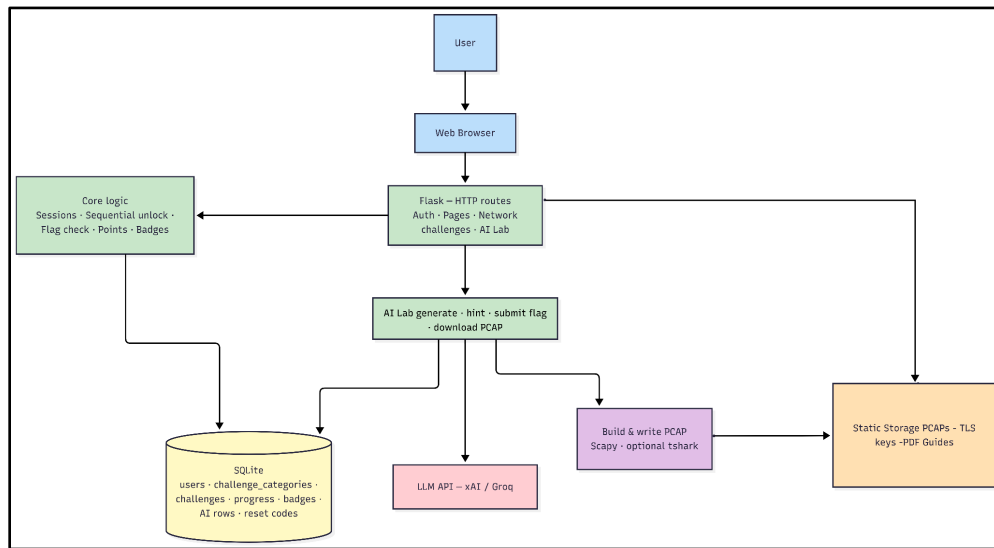


Figure 3: System Architecture Diagram

The overall system architecture of Thaghras and interaction of the major components of the system are shown in Figure 3. The platform is accessed via a web browser, linked to the Flask-based application layer, which implements authentication, routing, fixed challenges, and the AI Lab. Session handling, challenge unlocking and flag validation is handled by a core-logic layer, and the persistent data like progress, badges, the AI-generated metadata reside in the SQLite database.

In AI scenarios, the AI Lab interacts with external API providers to create challenge data based on natural-language prompts and then use Scapy, optionally including tshark, to generate packet captures for download. The predefined PCAP files, TLS keys and PDF guides are managed separately as static resources. In conclusion, the architecture showcases how the web interface, back-end logic, AI generation pipeline, packet generation workflows, and persistent storage are interconnected within Thaghrach.

4.3.2 Entity Relationship Diagram

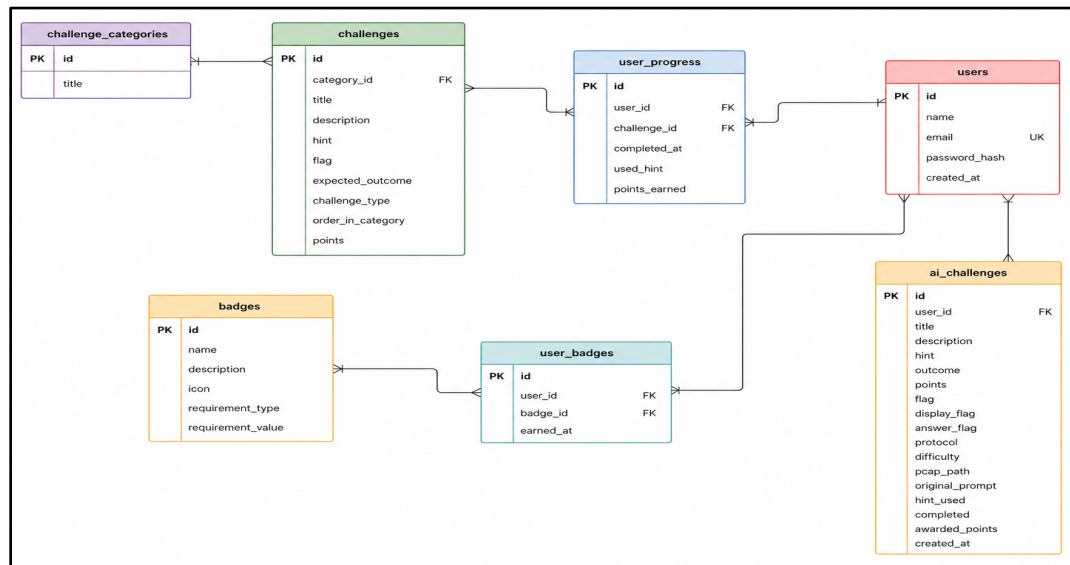


Figure 4: ERD Diagram

The Entity Relationship Diagram (ERD) of Thaghrach is illustrated in Figure 4, which displays the primary entities in the database and how they are connected. It is designed around the *users* table, which holds account details and links to user progress, badges earned, and AI generated challenge records. Fixed packet-analysis challenges are created via the *challenge_categories* and *challenges* tables, which contain challenge categories, descriptions, hints, flags, scoring, and other metadata.

The *user_progress* table stores the progress of the learner, including the challenges they have completed, hints used, points gained, and the time of completion. The *badges* and *user_badges* tables control the awarding of badges. The *ai_challenges* table contains all the information needed to create a dynamic scenario such as prompts, PCAP file path, flags, protocol information, scoring information, and completion.

Chapter 5

System Design

5.1 Introduction

This chapter introduces the design of Thaghrah, namely, the database schema, user interface design, system navigation flow, and procedures and algorithms that support the primary functions of the platform. The chapter describes the design of the proposed system to satisfy the requirements identified above and to offer a helpful cybersecurity learning experience.

5.2 Database Schema Design

5.2.1 Purpose of the Schema in the System

The database schema was designed to store the curriculum structure, learner accounts, challenge progression, scoring data, hint usage, and badge achievements in a structured relational format, as well as AI-generated challenge records. It also contributes to the flow of the progressions and validation processes deployed across the platform.

Foreign-key relationships and uniqueness constraints were added to ensure consistency of data across related tables and to facilitate consistent database initialization during development and testing (Elmasri, 2016). The schema also helps create a more maintainable system as it keeps challenges, scoring workflows and progress tracking all in one place, especially for features that rely on multiple connected tables like progression and badge a awarding logic.

5.2.2 Data Dictionary

Table 4: Users Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key; identifies the user in sessions and foreign keys.
name	TEXT	—	Display name collected at registration.
email	TEXT	NOT NULL, UNIQUE	Login identifier; normalized to lowercase in application logic.

password_hash	TEXT	NOT NULL	Werkzeug password hash (not plaintext).
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Approximate account creation time.

The *users* table stores registered users of the platform, with authentication details including username, email, password hash, and account creation time. The table is also the primary reference resource linked with learner progression, badge records, and AI-generated challenges.

Table 5: Challenge Categories Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Category identifier; referenced by challenges.category_id.
title	TEXT	NOT NULL	Short label shown in the UI (protocol family name).

The *challenge_categories* table is used to group the fixed curriculum by protocol. This will aid the organization of challenges in the interface and progression across the platform.

Table 6: Challenges Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Challenge primary key; aligns with seeded global track identifiers.
category_id	INTEGER	NOT NULL, FK → challenge_categories(id)	Links the challenge to its protocol category.
title	TEXT	NOT NULL	Challenge title displayed in the dashboard and challenge page.
description	TEXT	NOT NULL	Learner-facing challenge scenario and instructions.
hint	TEXT	NOT NULL	Optional guidance text; hint usage affects scoring logic.

flag	TEXT	NOT NULL	Expected flag value used during validation.
expected_outcome	TEXT	NOT NULL	Learning objective or intended challenge outcome.
challenge_type	TEXT	NOT NULL	Classification type used for filtering and organization.
order_in_category	INTEGER	NOT NULL	Ordering position within the category progression.
points	INTEGER	NOT NULL, DEFAULT 100	Maximum obtainable score before hint reduction.

The *challenges* table stores all challenges available in Thaghras. It includes descriptions of challenges, flags, hints, scoring details, expected results and category relationships for challenge progression and validation.

Table 7: User Progress Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key for the progress record.
user_id	INTEGER	NOT NULL, FK → users(id)	Learner who completed the challenge.
challenge_id	INTEGER	NOT NULL, FK → challenges(id)	Completed curriculum challenge.
completed_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of recorded completion.
used_hint	INTEGER	NOT NULL, DEFAULT 0	Indicates whether a hint was used during solving.
points_earned	INTEGER	NOT NULL, DEFAULT 0	Points awarded for the completed challenge.

The *user_progress* table stores completion information for fixed challenges. It records challenge completion, points, usage of hints, and completion time. This table is important for progression tracking and unlocking of sequential challenge.

Table 8: Badges Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Badge identifier referenced by user_badges.badge_id.
name	TEXT	NOT NULL	Badge title displayed in the interface.
description	TEXT	NOT NULL	Explanation shown to learners about the achievement.
icon	TEXT	NOT NULL	Icon or emoji token used for display purposes.
requirement_type	TEXT	NOT NULL	Badge rule category used by the awarding logic.
requirement_value	INTEGER	NOT NULL	Threshold value associated with the badge rule.

The *badges* table stores the achievement definitions used by the gamification system. Badge metadata, descriptions, display information, and rule conditions are all managed through this table.

Table 9: User Badges Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key for the awarded badge record.
user_id	INTEGER	NOT NULL, FK → users(id)	Learner who earned the badge.
badge_id	INTEGER	NOT NULL, FK → badges(id)	Badge definition associated with the award.
earned_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the badge was awarded.

The *user_badges* table associates users with the badges they collect. It preserves the connection between users and achievement records and facilitates the process of awarding badges.

Table 10: AI Challenges Table Metadata

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Surrogate key for the AI challenge record.
user_id	INTEGER	NOT NULL, FK → users(id)	Learner who generated the challenge.
title	TEXT	NOT NULL	Generated challenge title.
description	TEXT	NOT NULL	Scenario and instructions shown to the learner.
hint	TEXT	NOT NULL	Hint text associated with the generated challenge.
outcome	TEXT	NOT NULL	Expected learning outcome of the scenario.
points	INTEGER	NOT NULL, DEFAULT 100	Maximum obtainable points for the challenge.
flag	TEXT	NOT NULL	Stored flag used during validation.
display_flag	TEXT	—	Optional learner-facing or formatted flag representation.
answer_flag	TEXT	—	Canonical answer form used internally during validation.
protocol	TEXT	—	Protocol context associated with the generated scenario.
difficulty	TEXT	—	Difficulty label generated for the challenge.

pcap_path	TEXT	NOT NULL	Relative path to the generated PCAP file.
original_prompt	TEXT	NOT NULL	Natural-language prompt entered by the learner.
hint_used	INTEGER	NOT NULL, DEFAULT 0	Indicates whether the learner used a hint.
completed	INTEGER	NOT NULL, DEFAULT 0	Indicates whether the challenge was completed successfully.
awarded_points	INTEGER	NOT NULL, DEFAULT 0	Points awarded after challenge completion.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of challenge generation.

The *ai_challenges* table stores metadata related to dynamic challenges created in the AI Lab. This includes prompts, generated challenge information, PCAP paths, flags, metadata, completion status, and scoring information required for AI-driven workflows.

5.3 User Interface Design

5.3.1 Login and Registration Interface

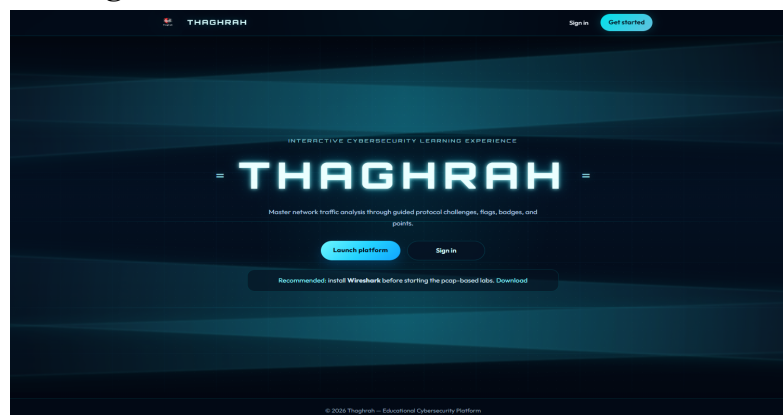


Figure 5: Welcome Page

THAGHRAH

Sign in Get started

NEW OPERATOR

CREATE ACCOUNT

Start your cybersecurity learning journey today.

Name

Your name

Email

you@example.com

Password

At least 8 characters, one capital letter, and one special character.

Confirm password

Create account

Already have an account? Sign in

Figure 6: Registration Page

THAGHRAH

Sign in Get started

SECURE ACCESS

SIGN IN

Welcome back. Sign in to continue your protocol training.

Email

you@example.com

Password

Sign in

Don't have an account? Register

Figure 7: Login Page

Figures 5-7 show the welcome, registration, and login interfaces for ThaghrAh. The welcome page introduces the platform and provides navigation options for the authentication pages. The registration page allows new learners to create accounts by entering personal and authentication information. The login interface allows registered users to access the platform securely using email and password credentials. After successful authentication, users are redirected to the dashboard.

5.3.2 Home Dashboard Interface

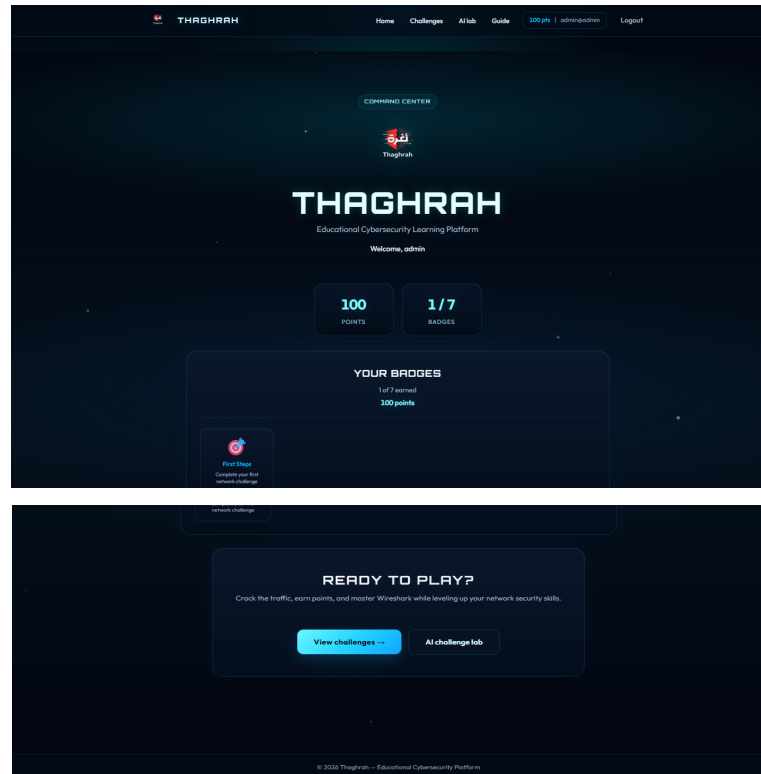


Figure 8: Home Dashboard Page

Figure 8 shows the main dashboard interface of Thaghran. Once logged in, students access the dashboard, which displays their points, acquired badges, and activity on the platform. It also includes quick-access sections and navigation bar for easy navigation to fixed challenges, protocol guides, and the AI Lab. The dashboard is the primary navigation section, where students can seamlessly advance their learning.

5.3.3 Challenge Interface

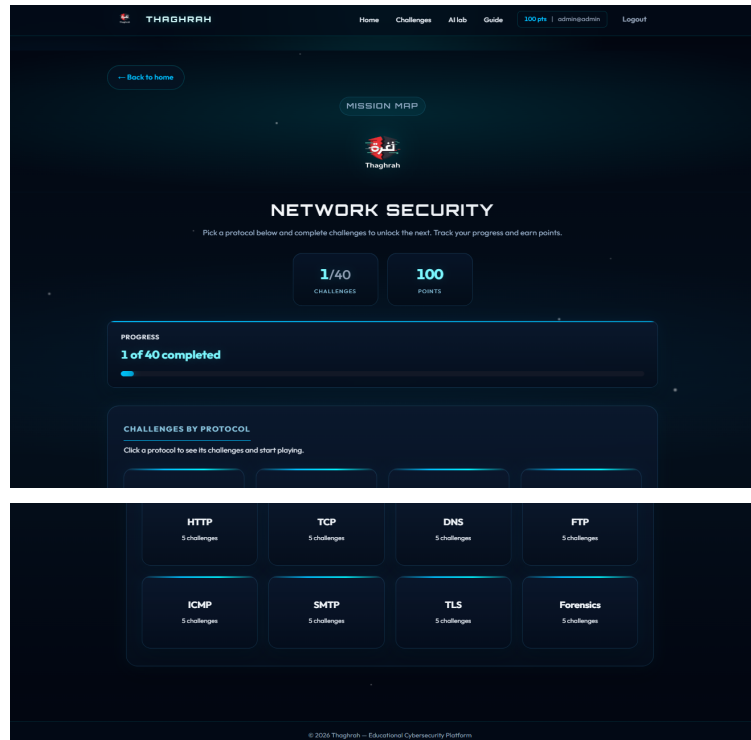
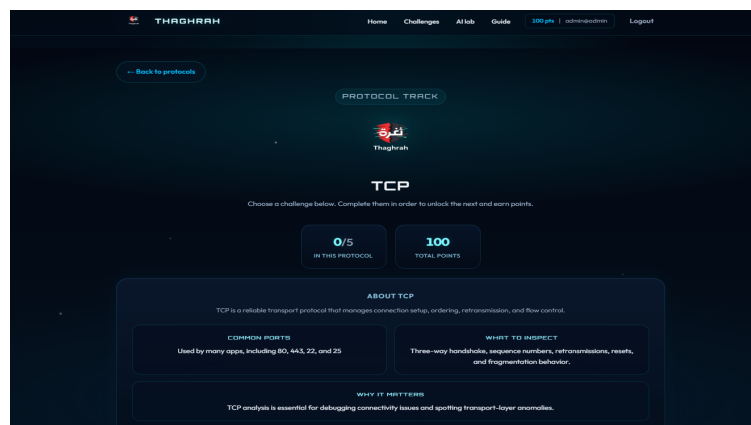


Figure 9: Challenge Navigation Page



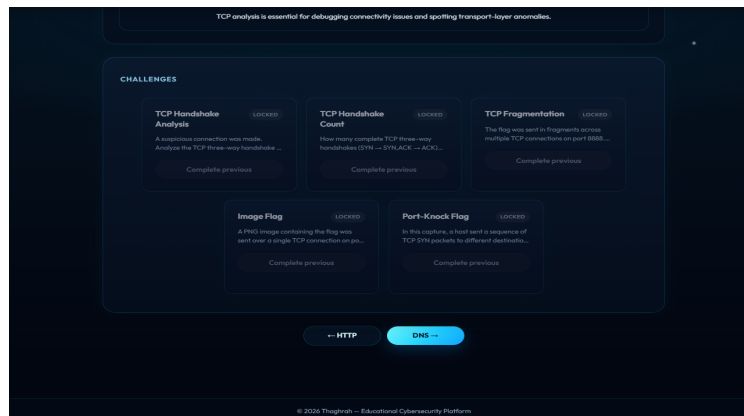


Figure 10: Protocol Challenges Page

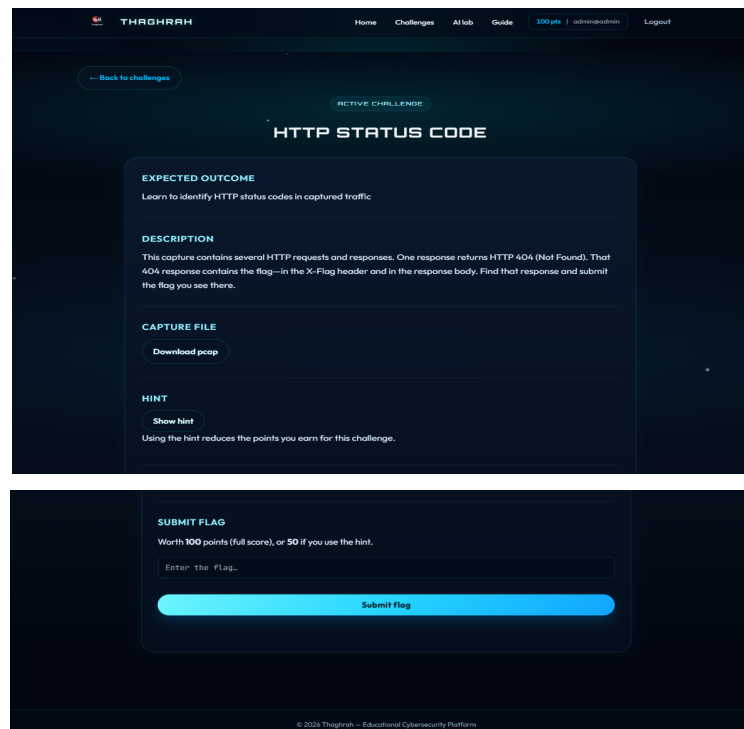


Figure 11: Challenge Page

Figures 9, 10 and 11 show the challenge interface which guides the learner from the mission map and protocol selection pages to the protocol tracks and detailed challenge pages. The mission map shows learner progress and protocol categories available, and each protocol page shows protocol information, focus areas, and specific challenges. The challenges are presented in order, with the learner having to finish the previous challenge before accessing the next. The detailed challenge

page contains the learning objective, description, PCAP file, optional hint, and a flag submission form for challenge validation and scoring. Navigation buttons on the screen are also added for better tracking between protocol tracks.

5.3.4 AI Lab Interface

The screenshot shows the 'AI LAB' section of the THAGHRAH application. At the top, there's a navigation bar with 'Home', 'Challenges', 'AI lab', and 'Guide'. A user profile section shows '100 pts | admin@admin' and a 'Logout' button. Below the navigation, a 'Home' button is visible. The main heading is 'AI PACKET CHALLENGE' with a subtext: 'Describe the scenario you want in plain language. The lab builds a unique PCAP, a learning goal, and a flag you can hunt in Wireshark.' Under 'YOUR PROMPT', examples are given: 'DNS exfiltration with Base64 chunks, medium difficulty' or 'HTTP login brute force in a small office network.' Below this, 'Ready Prompts:' are listed as 'Base64', 'Fragmentation', 'Encryption', and 'Bruteforce'. A text input field contains the placeholder 'Describe the challenge you want...'. A large blue button labeled 'Generate challenge' is at the bottom.

Figure 12: AI Challenge Page

This screenshot shows the same 'AI LAB' interface as Figure 12, but with an example prompt entered in the text field. The prompt is: 'ICMP traffic containing fragmented payloads that must be reassembled to recover hidden command instructions, hard difficulty'. The 'Generate challenge' button remains at the bottom.

Figure 13: AI Challenge Automatic Prompt Page

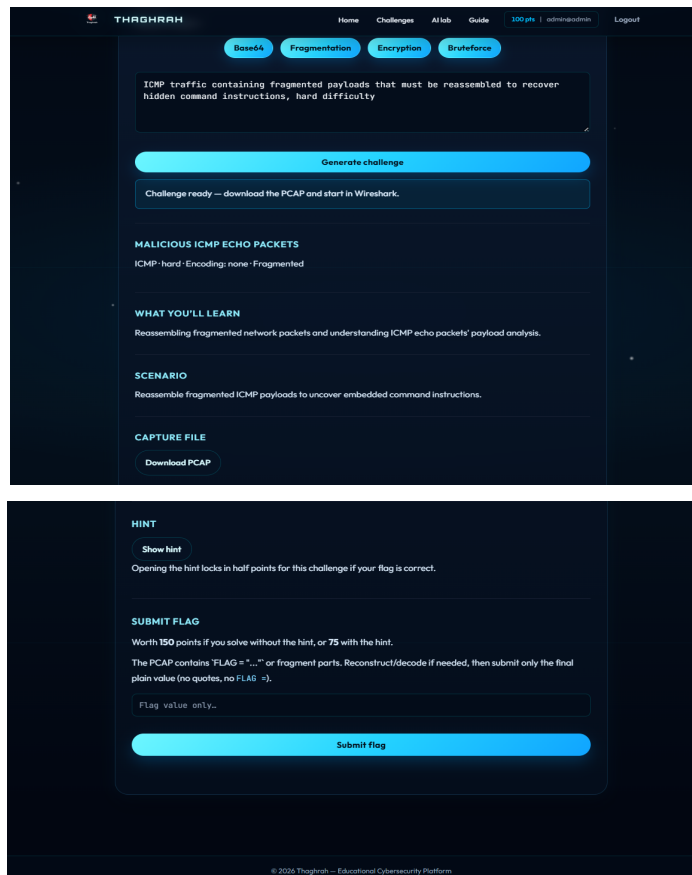


Figure 14: AI Challenge Generated Challenge Page

Figures 12, 13, and 14 illustrate the interface of the AI Lab, which enables the learner to create a scenario for packet-analysis by using natural-language prompts or suggested prompts like Base64, Fragmentation, Encryption, and Bruteforce. The generated challenges feature a title, description, learning outcome, downloadable PCAP file, hints, and flag submission.

5.3.5 Protocol Guide Interface

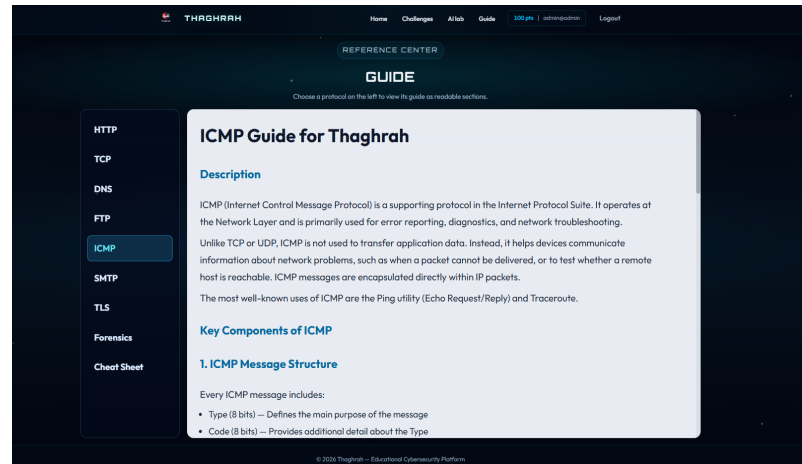


Figure 15: Guide Page

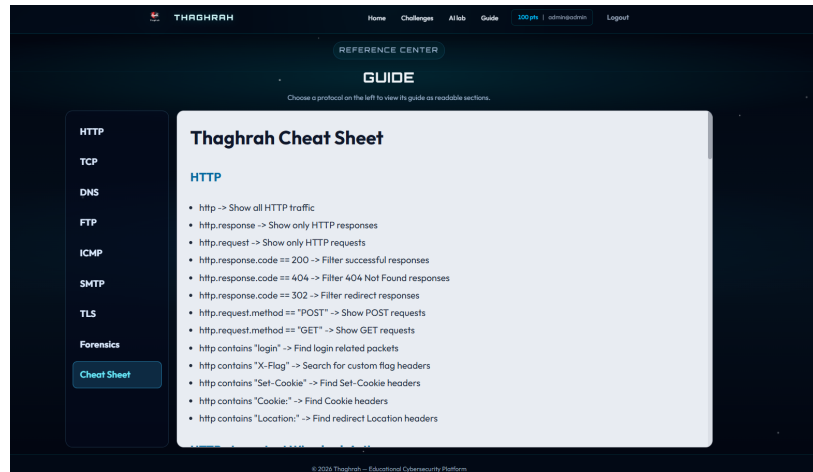


Figure 16: Cheat Sheet Page

Figure 15 shows the protocol-guide interface, which is an integrated learning reference that can be used for explanation and networking concepts in challenges. Users can select various protocol guides from the left-side navigation, each with a structured learning pathway and illustrations like pictures and previewed YouTube videos. There's also a cheat section as seen in Figure 16 with some of the most frequently used Wireshark display filters and packet-inspection commands.

5.3.6 Surprise Interface

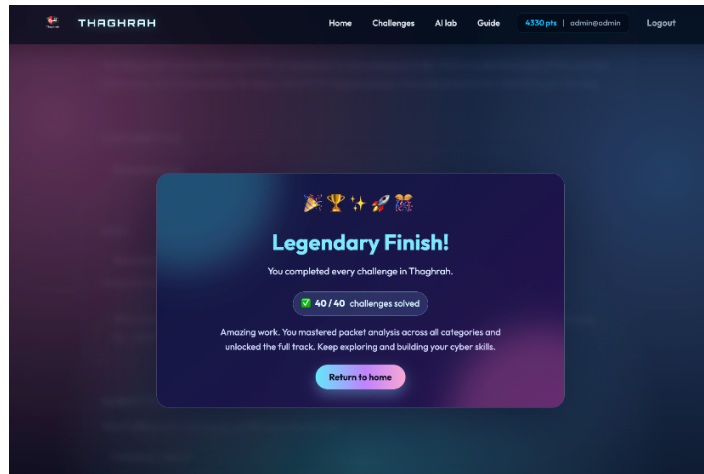


Figure 17: Surprise Page

5.4 Navigation and User Flow

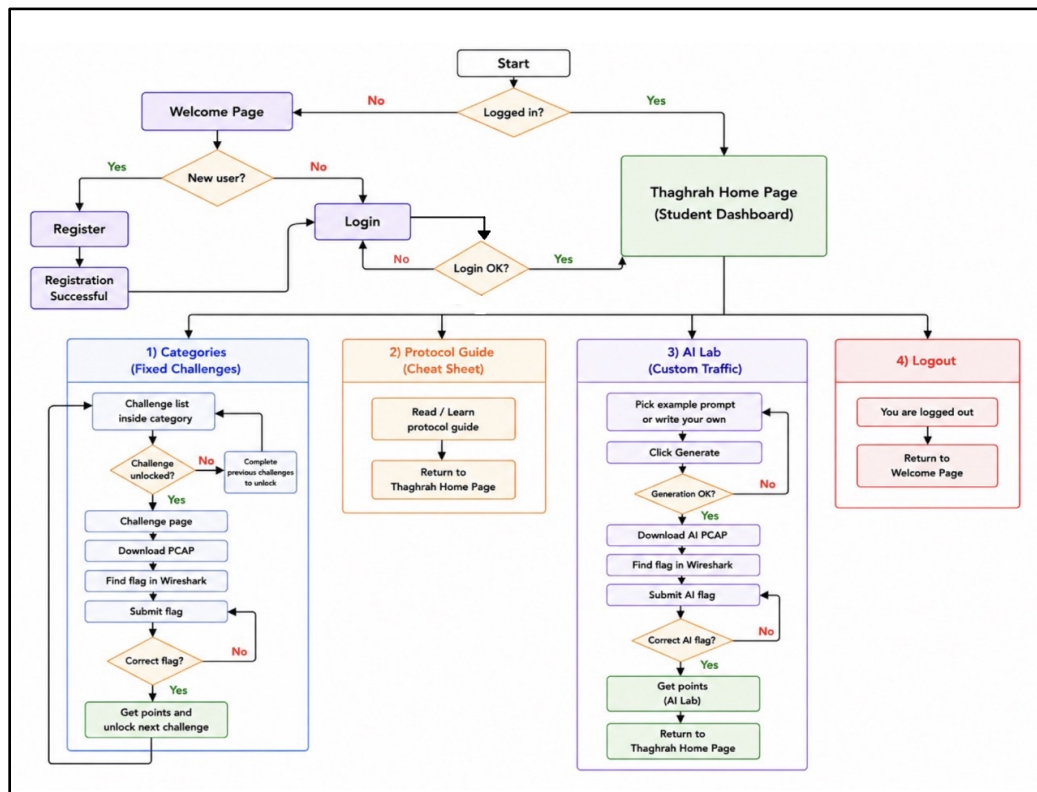


Figure 18: User Flow Diagram

The user flow diagram shows the interaction workflow in ThaghrAh. User authentication takes place via registration or login, then students log into the main dashboard of the platform. Users can go to fixed protocol challenges, protocol guides or the AI Lab from the dashboard. In the challenge workflow, learners receive a PCAP and after analyzing it with Wireshark, they submit extracted flags that are validated and scored. Users can dynamically create their own packet-analysis scenarios in the AI Lab. Additionally, the diagram shows additional navigation flows such as access to protocol guides and logout.

5.5 Procedure and Algorithm Design

5.5.1 AI PCAP Packet Construction

Pseudo code (pcap_plan.py):

```
FUNCTION build_packets_from_plan(plan, flag, answer, fragmentation,
fragment_count, encoding):
    Check that Scapy is available
    Clean and prepare the flag
    Get packet definitions from the plan
    Create an empty packet list

    For each packet definition:
        Read IPs, protocol, ports, and payload
        Remove old flag, answer, XOR, or fragment artifacts
        If fragmentation is OFF:
            Inject the full flag into the selected packet
            If XOR encoding is enabled:
                Inject the XOR key with the flag
        Build the packet using ICMP, DNS, UDP, or TCP
        Add the packet to the final list

    If fragmentation is ON:
        Split the flag into multiple parts
        For each fragment:
            Create a fragment packet
            Add fragment metadata and fragment data
            Add the packet to the final list

    If no packet contains the flag:
```

```

        Create a backup packet containing the flag
    If XOR encoding is enabled:
        Guarantee at least one packet contains both:
            XOR_KEY=23 and the flag
        Return the final packet list

```

5.5.2 AI Challenge Generation

Pseudo code (ai_lab.py)

```

Function ai_challenge_generate():
    Read and validate the user prompt
    If prompt is empty:
        Return error

    Generate challenge data using the AI model
    If generation fails:
        Return error

    Extract and validate flags from the response
    If flags are invalid:
        Return error

    Read protocol, encoding, encryption, fragmentation, and bruteforce
    settings

    Resolve fragmentation rules and fragment count

    Resolve encoding rules:
        Force XOR for encryption
        Apply requested encoding
        Auto-fallback to Base64 if needed

    Decode or re-encode flags depending on encoding

    Normalize the final answer flag
    Build the final display flag

    If encryption is enabled:
        Inject XOR key into the packet plan

    If bruteforce mode is enabled:
        Build bruteforce packet sequences

    Build packets from the PCAP plan

```

```

If packet creation fails:
    Return error

Generate challenge hints

Save the PCAP file
If saving fails:
    Return error

Save challenge metadata into the database
If database save fails:
    Delete PCAP and return error

Return successful JSON response

```

5.5.3 Calling LLM and Validating Challenge Payload

Pseudo code (grok_client.py):

```

FUNCTION call_grok_for_challenge(user_prompt):

    Load API key
    If missing:
        Return (False, user-safe error message, None)

    Choose API endpoint and model
    Validate prompt
    If empty or too long:
        Return (False, prompt error message, None)

    Build request payload using:
        System instructions prompt
        Provider-specific token limits/options
        JSON mode when supported

    Send request to AI service

    If HTTP request fails:
        Try recovery when possible:
            Repair or salvage partial JSON
            Regenerate structured JSON
            Retry with curl for certain Cloudflare blocks
        If recovery fails:
            Return (False, friendly error message, None)

    Parse successful raw response as API JSON

```

```

Read choices[0].message.content
If content is missing:
    Return (False, incomplete response message, None)

Extract challenge JSON from the content
If extraction fails:
    Try repair, regeneration, or best-effort recovery

Normalize text consistency
Force protocol to match the prompt

Validate the final challenge payload
If validation fails:
    Retry regeneration and fallback recovery
    If still invalid:
        Return (False, validation error message, None)

Return (True, "", payload)

```

5.5.4 Flag Validation

Pseudo code (challenge_utils.py):

```

FUNCTION network_flag_match(stored, submitted):

    Prepare the submitted answer
    If submitted answer is empty:
        Return False

    Normalize either:
        The inner FLAG = "..." value, if it exists
        Or the full submitted string

    If normalized submitted flag is empty:
        Return False

    Prepare the stored flag
    If stored flag is bytes:
        Decode it to text

    Create normalized stored candidates:
        Normalized full stored value
        Normalized inner stored flag value, if available

    If normalized submitted flag matches any candidate:
        Return True

    Return False

```

5.5.5 Flag Normalization

Pseudo code (challenge_utils.py):

```
FUNCTION normalize_network_flag(value):

    If value is None:
        Return ""

    If value is bytes:
        Decode it as UTF-8 (ignore errors)

    Convert value to string
    Trim whitespace

    Remove:
        Carriage returns (\r)
        Newlines (\n)
        Hidden Unicode characters
            (BOM and zero-width markers)

    Convert to uppercase

    Return the normalized result
```

5.5.6 Challenge Unlock Progression

Pseudo code (challenge_utils.py):

```
FUNCTION get_unlocked_challenges(challenges, completed_ids):

    Create an empty unlocked list

    For each challenge in order (index i):
        If i == 0
            OR previous challenge's ID is in completed_ids:
                Append this challenge's ID to unlocked

    Return unlocked
```

5.5.7 Flag Submission

Pseudo code (network.py):

```
FUNCTION submit_flag_route():

    Start timer

    Read JSON input:
        challenge ID
        submitted flag
        whether hint was used

    If challenge ID or flag is missing:
        Return failure message

    Trim submitted flag
    If flag is empty:
        Return failure message

    Convert challenge ID to a valid positive number
    If ID is invalid:
        Return failure message

    Map display ID to internal challenge ID
    If ID is unmapped:
        Return failure message

    Open database connection
    Fetch the challenge

    If challenge does not exist:
        Close database
        Return failure message

    Compare submitted flag with stored flag

    If flag is correct:
        Get base points from challenge
        If hint was used:
            Award half points
    Else:
        Award full points

    Check if user already completed this challenge
    If not completed:
        Mark challenge as completed
        Check and award new badges
```

```

    Calculate user progress:
        completed challenges
        total network challenges
        whether all are completed

    Close database
    Log successful submission

    Return success response with:
        message
        earned points
        new badges
        progress

Else:
    Close database
    Log incorrect submission
    Return incorrect flag message

```

5.5.8 Challenge Completion

Pseudo code (queries.py):

```

FUNCTION complete_challenge(conn, user_id, challenge_id, used_hint=False,
points_earned=0):

    Check if the user already completed the challenge

    If not completed:
        Convert used_hint into a database flag

        Insert a new progress record containing:
            user ID
            challenge ID
            hint usage
            earned points

        Save changes to the database
        Return True

    Return False

```

5.6 Summary

This chapter discussed the design of Thaghrah, including the database structure, user-interface design, navigation workflows, backend procedures, and algorithm logic used throughout the platform. It also explained the organization of packet-analysis challenges, AI-generated scenario workflows, scoring systems, and progression mechanisms connected to the platforms design.

Together, these components form the foundation used later during implementation and testing. Some workflows changed during development as more testing was performed, especially in areas connected to AI-generated traffic behavior and challenge validation.

Chapter 6

System Implementation and Testing

This chapter details the implementation and evaluation of Thaghrah, covering the development environment, technologies employed, structure of the project and testing processes that were followed during project development. It also includes evaluation outcomes for challenge workflow, security behavior, performance observations and AI-generated challenge functionality.

6.1 System Implementation

This section explains the implementation details including the development environment, selected technologies, and overall project structure used throughout the platform.

6.1.1 Development Environment and Hardware Specifications

The development and testing of Thaghrah were conducted in a single local workstation to ensure a stable and controlled environment in the entire project lifecycle. This gave a consistent environment for both the hardware and software as the platform relies on backend processing, Scapy, and external communication to the AI-provider which can be impacted by differences in the environment.

The platform has been developed on Python 3.13.3 and the dependencies are listed in the requirements.txt file. The Flask application was run using the command ``python run.py``, with the development server running on port 5001 while programming and testing it. The platform interaction and PCAP analysis were conducted using a modern web browser and Wireshark, with a few instances of using tshark and dumpcap for testing traffic-generation workflows.

Table 11: Hardware Configuration (Development and Testing Workstation)

Component	Specification
Operating system	Microsoft Windows 11 Home (Version 10.0, build 26200)
Processor	AMD Ryzen 7 3750H with Radeon Vega Mobile Gfx
RAM	16 GB
Primary storage	Micron 2200 NVMe SSD (512 GB class; ~477 GB reported capacity)

Graphics	NVIDIA GeForce GTX 1650; AMD Radeon RX Vega 10 Graphics (integrated)
----------	--

6.1.2 Technology Stack Selection and Justification

The technology stack was carefully designed to be lightweight, self-contained, and easily deployable in educational settings. The project design is a monolithic Flask-based application that handles routing and database access in a single application. The scenarios generated by AI are not built locally with models but rather through the APIs of external providers, thus lowering the requirements for hardware and avoiding dedicated GPUs.

Flask was selected in order to use its light-weight routing and session management features, plus its template rendering capabilities, and SQLite for its simple deployment locally and for persistent storage of user accounts and other metadata stored. Many challenges involved direct reliance on traffic behaviour, and Scapy was a major tool for building packets.

The frontend was developed with Jinja2 templates, HTML, CSS, and JavaScript, without making use of any other frontend framework to keep things simple. For communicating with external providers like xAI Grok and Groq, AI integration relied on the use of Python utilities to make HTTPS requests to OpenAI.

Table 12: Technology Stack Overview

Layer	Technology	Version / Notes
Language	Python	3.13.x / 3.12.x
Web framework	Flask	3.0.0
HTTP toolkit / crypto helpers	Werkzeug	3.0.1
Configuration	python-dotenv	≥ 1.0.0
Templating	Jinja2 (via Flask)	Bundled with Flask
Database	SQLite (sqlite3)	Embedded relational database

Authentication	Werkzeug password hashing; Flask sessions	<code>generate_password_hash</code> , <code>check_password_hash</code>
Packet manipulation / PCAP authoring	Scapy	$\geq 2.5.0$
Imaging (supporting assets)	Pillow	$\geq 10.0.0$
LLM integration (AI Lab)	HTTPS + JSON (<code>urllib</code> , <code>ssl</code>)	OpenAI-compatible endpoints
Client	HTML, CSS, JavaScript	Static files and rendered templates

6.1.3 Project Structure and Module Organization

```

Senior-Project/
├── run.py
├── schema.sql
├── requirements.txt
├── README.md
├── .env.example
├── .gitignore
├── thaghrdb.db
├── thaghrdb/
│   ├── __init__.py
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py
│   │   └── constants.py
│   ├── db/
│   │   ├── __init__.py
│   │   ├── database.py
│   │   └── queries.py
│   └── auth/
│       ├── __init__.py
│       └── decorators.py
└── # Entry: init_db(), Flask dev server (port 5001)
    # SQLite schema
    # Python dependencies
    # Description, features
    # API / AI env template (copy to .env)
    # Ignore secrets, venv, DB, logs
    # SQLite DB after first run (Runtime)

    # App package: factory, routes, domain, DB, AI
    # create_app(): logging, timing, AI PCAP cleanup, routes

    # Paths, .env, constants
    # Package init
    # Paths; load .env
    # e.g. PROTOCOL_NAMES (8 categories)

    # SQLite init + queries
    # Package init
    # get_db(), init_db() from schema.sql + seed
    # SQL helpers

    # Session guards
    # Package init
    # login_required

```

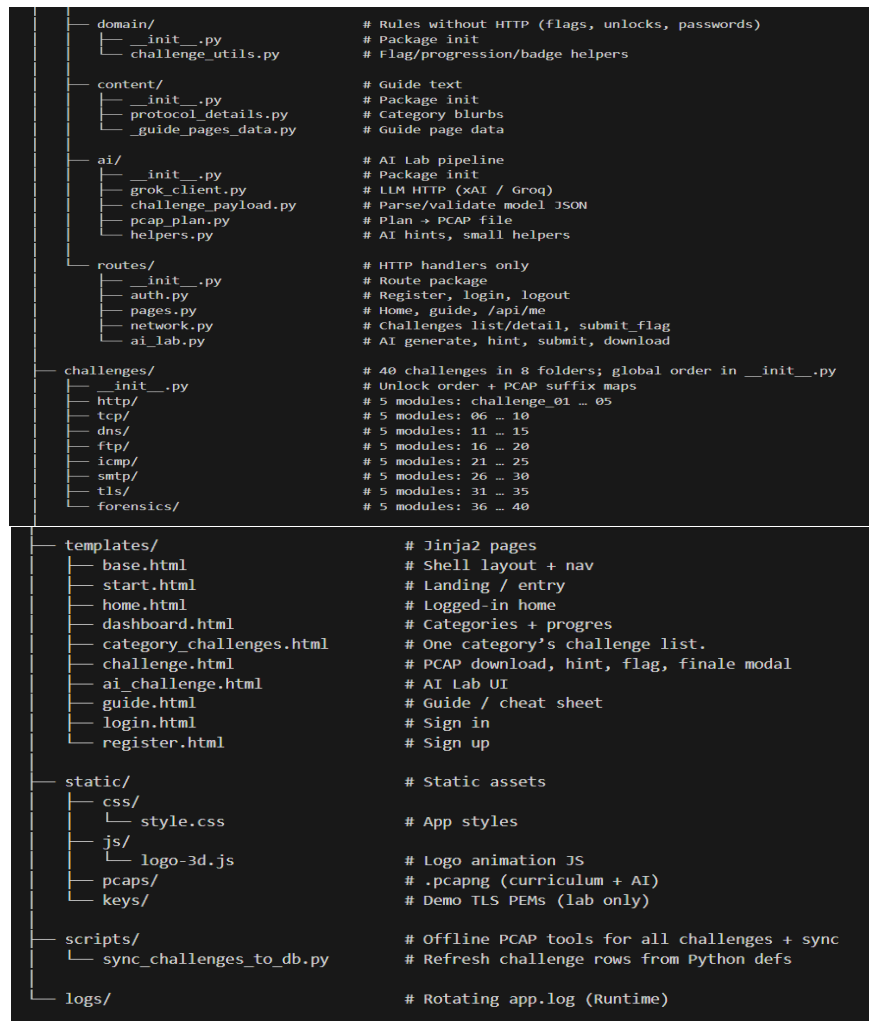


Figure 19: Project Structure

6.2 Evaluation and Testing

6.2.1 Testing Methodology

Thaghras was evaluated through a comprehensive testing process in a controlled local setting, considering functional workflows, security behavior, consistency of challenges generated by the AI, and the overall responsiveness of the system. Functional testing was verified for workflows, awarding and AI generations, and repeated testing to validate packet captures generated matched user prompts and to ensure that the generated packet captures supported valid challenge workflows.

Security testing covered security issues such as challenge access and JavaScript injection attempts. Although many aspects worked as intended, there were multiple revisions to various components, especially aspects of AI related behaviors, for inconsistencies found during testing.

6.2.2 Functional Testing

Functional testing was performed to ensure that the major processes performed properly in the education setting. This also involved validating that every challenge contained the right PCAP file and that there was correct flag validation of both valid and invalid submissions.

User tests were conducted to verify that the scoring penalties for hints were applied as intended and the badge awarding function was tested by completing the progression milestones and checking the score awards were correct. AI-generated challenges were also tested, with varying prompts, to ensure that generated scenarios resulted in valid PCAP files, appropriate hints, and valid challenge scenarios matched to the desired type of investigation.

Table 13: Functional Testing Results

Test ID	Functional Area	Test Description	Expected Result	Result
FT-01	PCAP Download	Download the PCAP file associated with a challenge	Correct PCAP file downloads successfully	Pass
FT-02	Flag Validation	Submit incorrect and correct flags for a challenge	Incorrect flags rejected; correct flags accepted with validation message	Pass
FT-03	Hint Penalty System	Open challenge hint and submit correct flag	Reduced score awarded after hint usage	Pass
FT-04	Badge Awarding	Complete required challenge milestone	Badge notification displayed and reward assigned	Pass
FT-05	AI Challenge Generation	Generate AI-based packet-analysis challenge from user prompt	Valid challenge, PCAP file, hint, and flag validation generated according to prompt	Pass

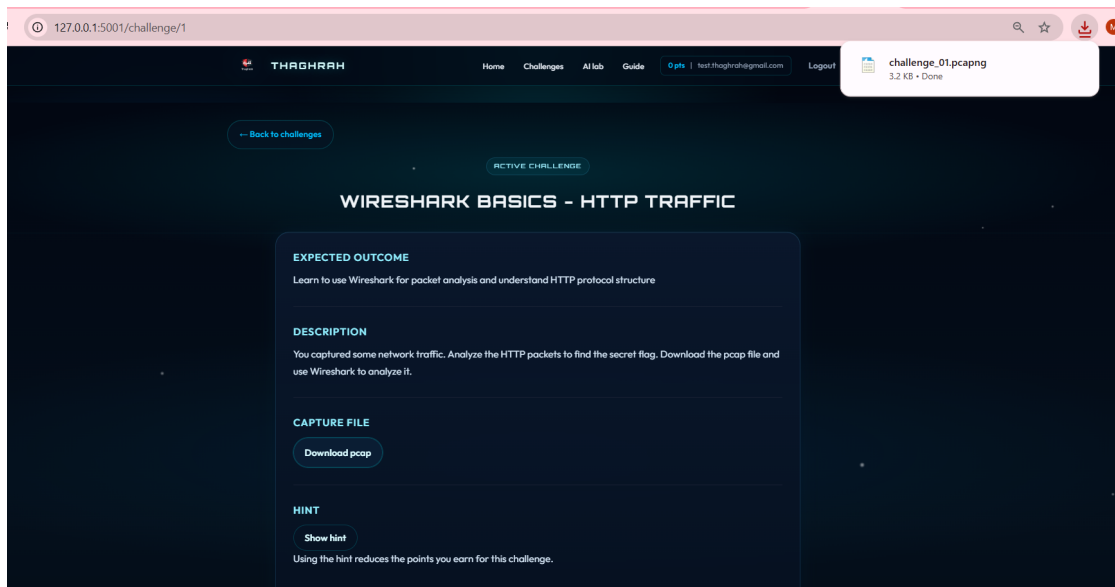


Figure 20: PCAP Download Validation

Figure 20 demonstrates successful PCAP download functionality through the challenge interface. The correct downloadable capture file was linked to the selected challenge scenario.

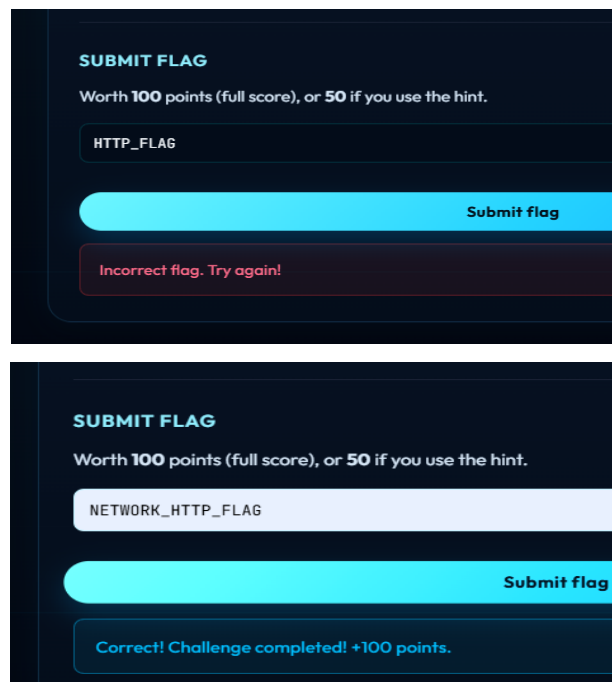


Figure 21: Flag Validation Testing

Figure 21 illustrates backend-controlled flag validation using both incorrect and correct submissions. Invalid answers were rejected while valid flags produced confirmation feedback and challenge completion.

The screenshot shows a dark-themed user interface for a challenge. At the top, a 'HINT' section contains a 'Show hint' button. Below the button, a message states: 'Using the hint reduces the points you earn for this challenge.' A text box contains the hint: 'Find the request to /missing; the response to that request is 404. In that response, look at the X-Flag header or the response body — the flag appears there.' Below the hint section is a 'SUBMIT FLAG' section. It states: 'Worth 100 points (full score), or 50 if you use the hint.' A text input field contains the flag 'STATUS_404'. Below the input field is a large blue 'Submit flag' button. At the bottom, a green message box says: 'Correct! Challenge completed! +50 points.'

Figure 22: Hint Penalty Validation

The figure demonstrates the hint-based scoring workflow implemented in the platform. Opening the hint reduced the awarded points from the full challenge value to half points after successful completion.

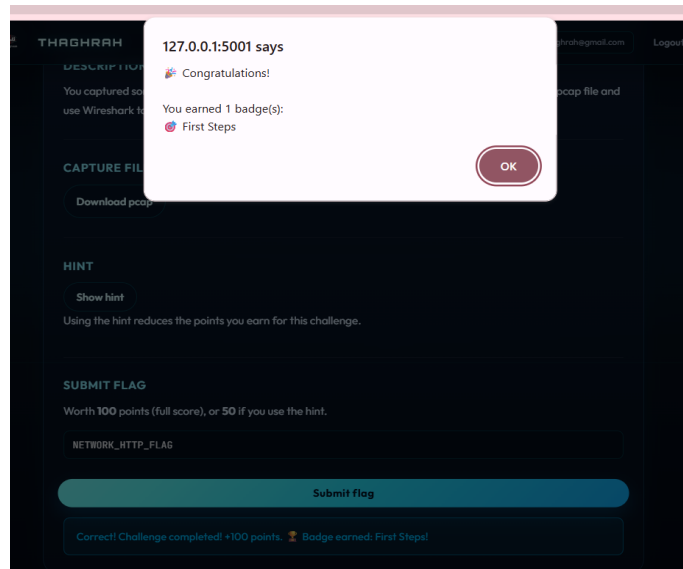


Figure 23: Badge Awarding Validation

Figure 23 shows successful badge assignment after the learner satisfied the required progression condition. The workflow confirmed that milestone rewards and badge notifications operated correctly inside the platform.

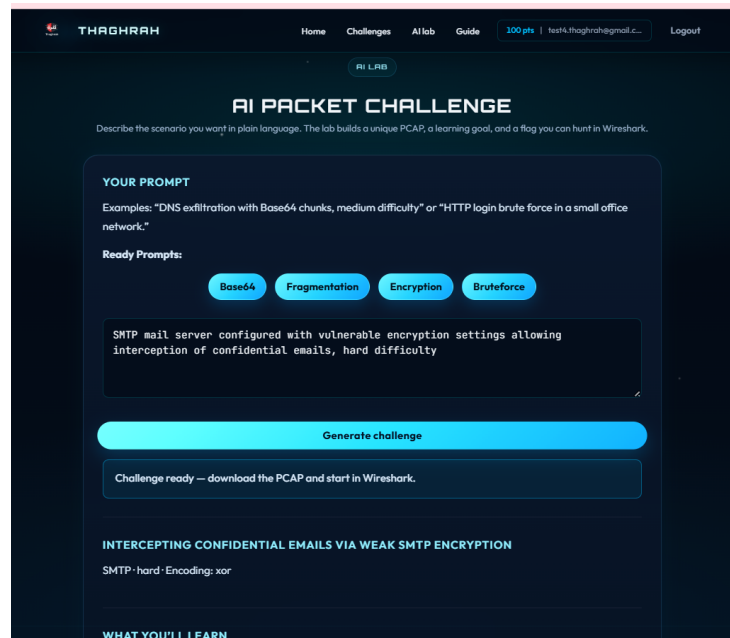


Figure 24: AI Challenge Generation Validation

The figure demonstrates successful AI-generated challenge creation using a user-provided prompt. The generated output included a downloadable PCAP file, challenge description, hint mechanism, and validation workflow corresponding to the entered investigation scenario.

6.2.3 Security Testing

Security testing was conducted to assess the security measures put in place. The educational environment was tested for password protection, input handling in the frontend, validation of authentication and restrictions on accessing the challenge.

The password workflow was verified, making sure the credentials were stored as hashed values, not plain text. JavaScript injection attacks were attempted on the frontend to ensure that malicious entries were not recognized as executable code. Challenge-access protection was tested by manually attempting to access locked challenge URLs to confirm that unauthorized users were not allowed to proceed. Weak passwords and invalid credentials were also used to test authentication validation and enforcement of password policies.

Table 14: Security Testing Results

Test ID	Security Area	Test Description	Expected Result	Result
SEC-01	Password Hashing	Verify password storage in database and logs	Passwords stored and logged as hashes only	Pass
SEC-02	XSS Protection	Inject JavaScript payload in registration name field	Input treated as plain text without execution	Pass
SEC-03	Locked Challenge Access	Manually access locked challenge URL	User redirected to allowed challenge page	Pass
SEC-04	Authentication Validation	Submit invalid login credentials	Validation message displayed and login rejected	Pass
SEC-05	Password Policy Enforcement	Submit weak password during registration	Password rejected until complexity rules are satisfied	Pass

	id	name	email	pass
1	1	Test	test.thaghr@gh@gmail...	script
2	2	<script>hello</scri...	test@gmail.com	script
3	3	hello	test2.thaghr@gh@gmail...	script
4	4	<script>echo "hello...	test4.thaghr@gh@gmail...	script
+				

Figure 28: XSS Input Handling Validation

The figures illustrate a JavaScript injection attempt inside the registration form. The input was treated as plain text and was not executed inside the application interface.

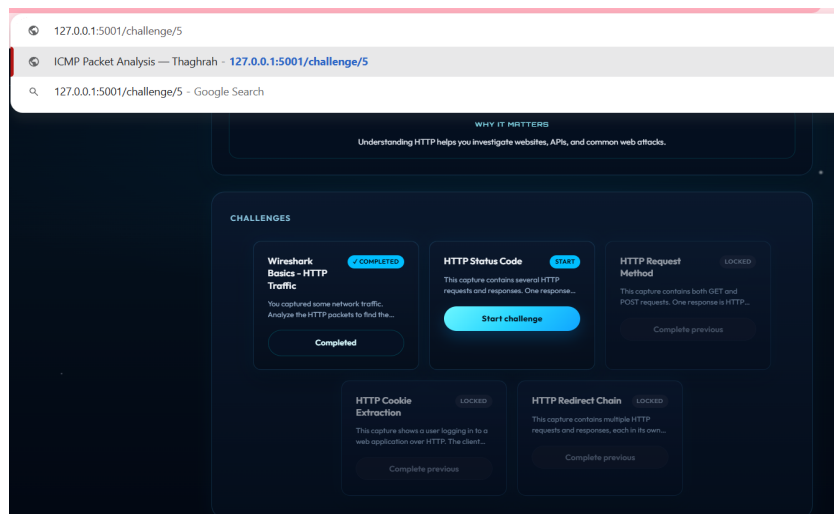


Figure 29: URL Access Attempt

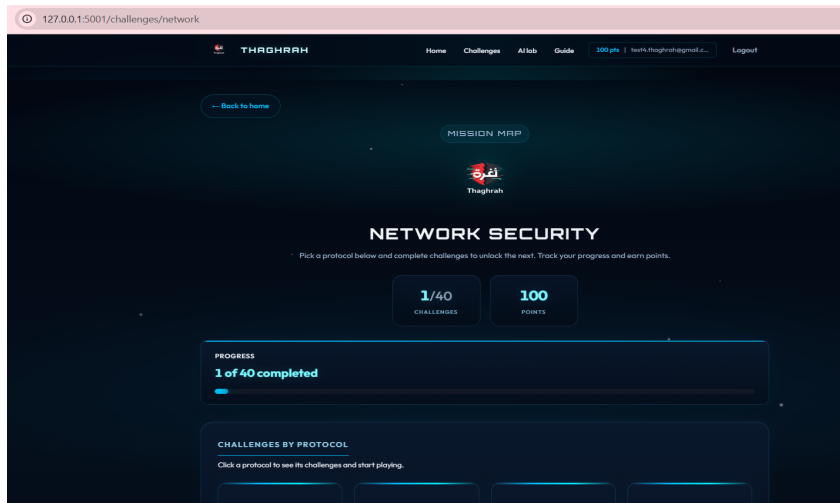


Figure 30: Locked Challenge Access Protection

The figures demonstrate an attempt to access a locked challenge directly through the URL. The platform redirected the learner to an allowed challenge page and prevented unauthorized progression bypass.

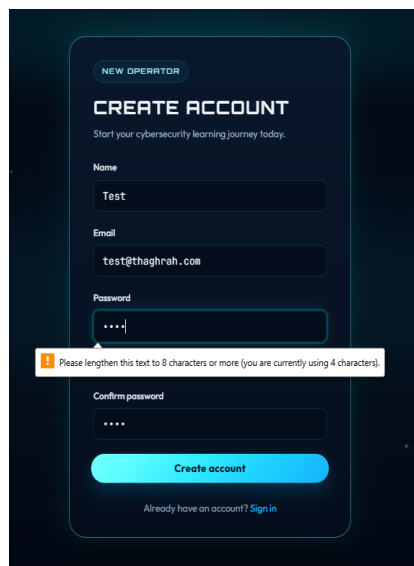


Figure 31: Weak Credentials Validation

NEW OPERATOR

CREATE ACCOUNT

Start your cybersecurity learning journey today.

Password must contain at least one special character.

Name
Your name

Email
admin@admin

Password

At least 8 characters, one capital letter, and one special character

Confirm password

Create account

Already have an account? [Sign in](#)

Figure 32: Weak Credential Validation

SECURE ACCESS

SIGN IN

Welcome back. Sign in to continue your protocol training.

Invalid email or password

Email
test4.thaghr@h@gmail.com

Password

Sign in

Don't have an account? [Register](#)

Figure 33: Weak Credential Validation

The figures demonstrate invalid-login handling and weak-password validation. Authentication checks and password complexity requirements operated correctly during testing.

Security workflows behaved consistently during testing, although several validation rules were adjusted during development after edge cases appeared involving malformed inputs and progression logic.

6.2.4 Performance Testing

Performance tests were used to measure the responsiveness of workflows under normal platform use. The key test areas were server and client submission-processing times. Backend performance was measured by examining application logs containing recorded `elapsed_ms` values, and browser developer tools were used to view the round-trip time of the requests that the users experienced.

The measurements were assessed separately because they are of different kinds of timing behavior. The results indicated that the challenge-validation workflow was consistent and had short processing time despite no significant delay occurring during the normal educational use in the testing environment.

Name	Status	Type	Initiator	Size	Time
 style.css	304	stylesheet	Other	0.3 kB	16 ms
 submit_flag	200	fetch	1:307	0.2 kB	13 ms

Figure 34: Browser Time

```

2026-05-11 21:41:15,854 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:15] "GET /challenge/1 HTTP/1.1" 200 -
2026-05-11 21:41:15,936 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:15] "esc[36mGET /static/css/style.css HTTP
2026-05-11 21:41:15,941 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:15] "esc[36mGET /static/images/logo.png HT
2026-05-11 21:41:15,969 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:15] "GET /api/me HTTP/1.1" 200 -
2026-05-11 21:41:26,734 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:26] "esc[36mGET /static/css/style.css HTTP
2026-05-11 21:41:26,802 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:26] "esc[33mGET /.well-known/appspecific/c
2026-05-11 21:41:44,679 INFO thaghras SUBMIT_FLAG incorrect challenge_id=1 user_id=5 elapsed_ms=3.66 submitted=1
2026-05-11 21:41:44,680 INFO thaghras REQ method=POST path=/submit_flag status=200 elapsed_ms=4.59 user_id=5
2026-05-11 21:41:44,681 INFO werkzeug 127.0.0.1 - - [11/May/2026 21:41:44] "POST /submit_flag HTTP/1.1" 200 -

```

Figure 35: Log Time

6.2.5 AI Accuracy Testing

AI accuracy testing was conducted to evaluate the consistency and correctness of AI-generated challenges inside Thaghrh. Ten different generation attempts were performed using cybersecurity-related prompts involving protocols, encoded payloads, fragmented traffic, and brute-force scenarios.

Each generated challenge was evaluated based on scenario relevance, generated flag quality, downloadable PCAP functionality, and consistency with the supplied prompt.

Most generated scenarios produced meaningful investigation workflows, coherent flags, and functional downloadable PCAP files. However, the fifth and sixth generation attempts produced unrealistic flag wording containing random underscore-separated values. Another generated challenge produced a flag with a missing character near the end of the value.

Despite these inconsistencies, the remaining attempts generated coherent scenarios and valid workflows matching the requested prompts.

Table 15: AI Accuracy Testing Results

Try No.	Prompt	Result	Description
1	DNS traffic containing Base64-encoded employee credentials hidden inside outbound queries	Pass	Accurate challenge generation with meaningful flag and valid PCAP workflow
2	ICMP traffic containing fragmented payloads that reconstruct hidden administrative credentials	Pass	Generated challenge and flag matched the supplied scenario correctly
3	SMTP mail server configured with vulnerable encryption settings allowing interception of confidential emails	Pass	Accurate scenario generation with coherent flag and downloadable PCAP
4	HTTP login portal targeted by repeated brute-force authentication attempts against administrator accounts	Pass	Challenge behavior and generated flag aligned with the prompt
5	FTP traffic with Base64-encoded authentication tokens transferred between client and server	Partial Pass	Challenge generated successfully, but flag value contained unrealistic random wording

6	Fragmented WhatsApp-style mobile traffic revealing hidden exfiltration behavior	Partial Pass	Generated challenge functioned correctly, but flag value lacked semantic meaning
7	FTP forensic challenge involving credential brute-force attacks against an exposed FTP server	Pass	Accurate challenge generation with valid flag and PCAP workflow
8	Medium-difficulty malware communication challenge using encoded HTTP traffic	Partial Pass	Scenario generated correctly, but generated flag contained a missing character
9	TLS traffic challenge involving fragmented Base64 credential transfer	Pass	Generated challenge and validation workflow behaved correctly
10	Brute-force login attempts against a target web application	Pass	Accurate challenge generation with coherent flag and downloadable PCAP

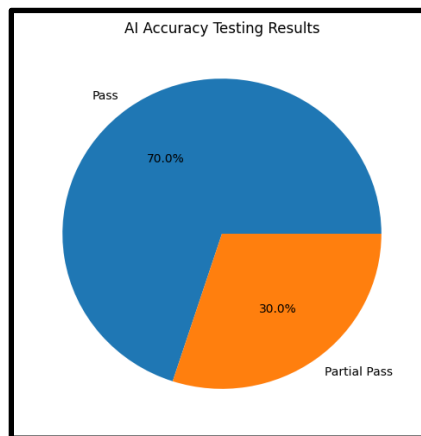


Figure 36: AI Accuracy Testing Results

6.2.6 Discussion, Comparison, Strengths, and Weaknesses

The results of the evaluation process indicated that Thaghrah is able to support the learning of packet analysis. During normal use, functional and security tests, along with performance tests, demonstrated reliable challenge interaction, validation, access control and workflow behavior.

Compared with platforms such as TryHackMe and OverTheWire, Thaghras focuses more specifically on Wireshark-based forensic analysis and packet-investigation rather than broader cybersecurity training environments. AI challenges also bring variations in the scenarios, unlike purely static platforms.

There were some limitations, however, found such as flags sometimes appearing inconsistent due to AI generation, reliance on an external AI provider API, and the SQLite architecture is more suited for small-scale education use rather than large-scale.

Table 16: Strengths of Thaghras

Strength	Description
Structured Learning Workflow	Sequential challenge progression supports guided packet-analysis learning
Realistic PCAP-Based Training	Downloadable PCAP files provide practical Wireshark investigation experience
AI-Generated Challenges	Dynamic challenge creation increases scenario diversity and replayability
Gamified Learning Features	Scoring, hints, and badge systems improve engagement and motivation
Beginner-Friendly Design	Simplified interface and guided workflows support new learners

Table 17: Weaknesses of Thaghras

Weakness	Description
AI Generation Inconsistency	Some generated flags lacked semantic consistency or contained formatting issues
External AI Dependency	AI Lab functionality depends on external API availability and response quality
Limited Scalability	SQLite and local deployment architecture are more suitable for educational-scale usage
No Real-Time Traffic Capture	The platform focuses on downloadable PCAP analysis rather than live packet interception

6.3 Summary

This chapter covered the implementation and evaluation of Thaghrah and its development environment, technologies, testing, and results. It described the functional, security, performance, and AI-accuracy tests that were used to validate the platform.

The findings indicated that the system of structured packet-analysis learning, using downloadable PCAP investigations, progressive learning paths and validation workflows with feedback, as well as AI tools for creating challenges is very helpful for learning in a structured way, while still maintaining an educational context. Some issues were noted, such as flag consistency and scalability of the platform, but these did not affect its use and it met its learning outcome goals for training beginners in packet analysis.

Chapter 7

Conclusion and Future Work

The project was successful in meeting the intended objectives, which included designing and developing a functional educational platform, providing structured cybersecurity challenges for hands-on learning, and generating scenarios with AI. Despite some existing constraints, especially regarding the consistency of the generated challenges for AI-driven generation and the limited range of features available, the project creates a solid foundation for future improvements and highlights the potential of integrating practical cybersecurity training with intelligent challenge-generating technologies.

There are also a number of enhancements that may be planned for the future. Additional cybersecurity domains such as malware analysis, web security, or digital forensics could be integrated to provide more variety for learners. The AI-generation pipeline can also be expanded to support additional scenarios, such as additional protocol types and investigation behaviors. Real-time packet analysis is another potential upgrade, since the platform is predominately based on PCAP workflows that can be downloaded, instead of real-time packet interception or traffic monitoring.

References

- Kolb, D., 1984. *Experiential Learning: Experience as the Source of Learning and Development*. Englewood Cliffs, NJ: Prentice Hall.
- Vygotsky, L., 1978. *Mind in Society: The Development of Higher Psychological Processes*. Cambridge, MA: Harvard University Press.
- Nielsen, J., 1994. *Usability Engineering*. City: San Francisco: Morgan Kaufmann.
- Pressman, R. a. M. B., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. s.l.:McGraw-Hill.
- Sommerville, I., 2016. *Software Engineering*. 10th ed. s.l.:Pearson.
- Grinberg, M., 2018. *Flask Web Development: Developing Web Applications with Python*. 2nd ed. s.l.:O'Reilly Media.
- Elmasri, R. a. N. S., 2016. *Fundamentals of Database Systems*. 7th ed. s.l.:Pearson.
- Myers, G. S. C. a. B. T., 2011. *The Art of Software Testing*. 3rd ed. s.l.:Wiley.
- Kasneci, E. et al., 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, Volume 103.
- Hamari, J., Koivisto, J. & Sarsa, H., 2014. Does gamification work? A literature review of empirical studies on gamification. *47th Hawaii International Conference on System Sciences*.
- Foundation, O., 2023. *Password Storage Cheat Sheet*. [Online]
Available at: <https://owasp.org>
- Myers, G., Sandler, C. & Badgett, T., 2011. *The Art of Software Testing*. 3rd ed. s.l.:wiley.

Orebaugh, A., Ramirez, G. & Burke, J., 2006. *Wireshark & Ethereal Network Protocol Analyzer Toolkit*. s.l.:Syngress.

Chapman, P., Burket, J. & Brumley, D., n.d. *PicoCTF: A game-based computer security competition for high school students Year: 2014*. s.l., s.n.

in, G. et al., 2018. *Game based cybersecurity training for high school students*. s.l., s.n.

PortSwigger, 2024. *Burp Suite Documentation*. [Online]
Available at: <https://portswigger.net/burp>